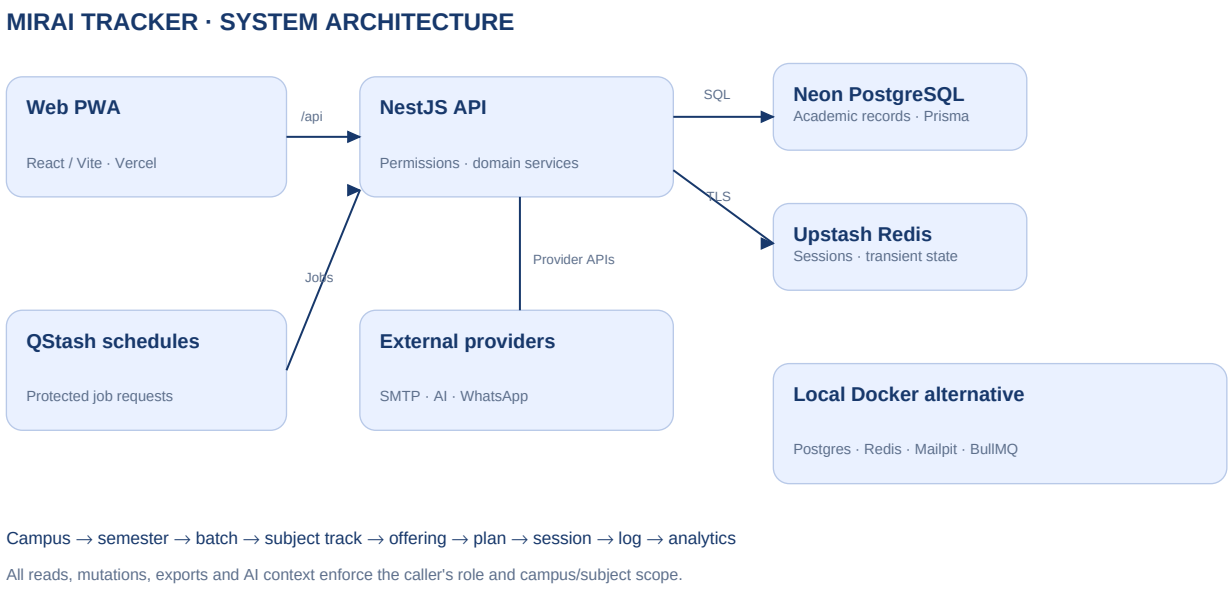
Architecture and design

MIRAI TRACKER · 20 OCTOBER 2026



Purpose and scope

Mirai Tracker connects planning to recorded teaching for HI-Tech, Jaggannath and Rathinam. A semester belongs to a campus. A batch contains offerings in Mirai and University tracks. Shared records drive dashboards, reports, faculty plans and the student view.

Deployment

React/Vite serves the PWA from Vercel. Same-origin /api requests reach NestJS on Vercel. Prisma accesses Neon PostgreSQL. Upstash Redis stores opaque sessions and transient auth/job state. QStash invokes protected scheduled endpoints. SMTP delivers OTPs and summaries; AI and WhatsApp are separately configured providers.

Local Compose runs the web, API, PostgreSQL, Redis, Mailpit and BullMQ worker. The load report uses a host API with local containerized data services. It does not claim to reproduce hosted network latency.

Data and authorization

Users have campus-scoped roles. Faculty profiles link to offerings through dated assignments. Students link to a batch. Campus → semester → batch → offering → lesson plan → unit → topic forms the academic hierarchy. Timetable entries generate sessions; revisioned logs record topic updates. Exam windows map to exact topics.

Administrators/program managers oversee all campuses. Campus managers manage their campuses. Faculty work with assigned subjects. Students read their enrolled batch. Server-side guards and row scoping apply to reads, writes, reports, exports and AI context.

Calculation and calendar

Topic minutes are canonical. Covered minutes include completed topics and weighted partial completion. Remaining minutes equal scope minus covered minutes. Available minutes come from eligible future sessions through the deadline. Buffer is available divided by remaining. Default thresholds are 1.15 for ON TRACK and 1.00 for TIGHT; complete and missing-plan states are distinct.

Explicit calendar entries override weekly rules. Sundays and non-working Saturdays are excluded unless opened. Timetable generation checks conflicts and preserves logged history. Changes produce revised planned dates; affected classes can require faculty review before a recovery move.

Imports, delivery and reports

CSV/XLSX rows are parsed, validated and previewed before domain writes. AI helps with unstructured calendars and ambiguous mappings. Separate pages import calendars, batch timetables, lesson plans and student access. The full-semester import interface is retired.

Logs record completion, classwork/homework and topic changes. Reports use shared scoped analytics. Exports retain structured records. Visiting-faculty honoraria derive from recorded classes and effective per-class rates, with expenses/payments shown separately.

Security and operations

Passwords use salted scrypt. OTPs expire after ten minutes with attempt/request limits. Session cookies are HTTP-only, SameSite and Secure in production. Authorization is refreshed from PostgreSQL. Validation, origin checks and safe errors protect mutations. AI receives scoped data and output is rendered without raw HTML execution.

Transactions protect related writes. Unique job/revision keys and leases reduce duplicate processing. Scheduler secrets protect internal routes. Audit records retain actor and scope. Secrets and demo passwords stay outside Git and Docker build contexts.

The scale fixture contains three campuses, 60 faculty, 30 subjects and 1,020 students. The separate report records 50/100/200-client loads, a 60-faculty burst and full-semester reads. Pool budgeting, permission-safe aggregation and hosted sustained tests guide further scaling.